

Paterni ponašanja

Strategy patern

Strategy patern koristi se kada postoji više različitih algoritama za rješavanje istog problema, pri čemu se konkretni algoritam može birati u zavisnosti od potrebe korisnika ili trenutnog konteksta. Umjesto da se svi algoritmi nalaze u jednoj klasi, oni se izdvajaju u posebne klase, čime se kod čini preglednijim i lakšim za proširenje.

U aplikaciji Biblioteka ovaj patern je pogodan za funkcionalnost pretrage i sortiranja kataloga knjiga. Korisnici mogu pretraživati i sortirati knjige prema različitim kriterijima, kao što su naslov, autor, žanr, godina izdanja ili prosječna ocjena. Svaki od ovih načina sortiranja može se izdvojiti kao posebna strategija. Na taj način bi se novi kriterij sortiranja mogao dodati bez izmjene glavne logike prikaza kataloga.

State patern

State patern omogućava da se ponašanje objekta mijenja u zavisnosti od njegovog trenutnog stanja. Umjesto da se u kodu stalno provjerava stanje pomoću velikog broja if ili switch naredbi, svako stanje može imati posebno definisano ponašanje.

Ovaj patern se prirodno uklapa u rad sa primjercima knjiga. Primjerak može biti dostupan, posuđen, rezervisan ili izgubljen. Od trenutnog stanja zavisi koje su akcije dozvoljene. Dostupan primjerak se može posuditi, posuđen primjerak se može vratiti ili eventualno produžiti, dok izgubljen primjerak ne bi trebao biti dostupan za nove posudbe. Korištenjem State paternna svako od ovih stanja moglo bi se jasno izdvojiti i zasebno obrađivati.

Template Method patern

Template Method patern definiše osnovnu strukturu određenog procesa, dok se pojedini koraci tog procesa mogu mijenjajući implementirati u izvedenim klasama. Time se izbjegava ponavljanje zajedničke logike, a istovremeno se ostavlja mogućnost prilagođavanja specifičnih dijelova algoritma.

U našoj aplikaciji ovaj patern se može primijeniti kod obrade različitih zahtjeva nad podacima. Naprimjer, dodavanje, izmjena i brisanje knjige imaju nekoliko zajedničkih koraka: provjeru korisničke uloge, validaciju podataka, pristup bazi i prikaz rezultata korisniku. Međutim, konkretan postupak se razlikuje u zavisnosti od toga da li se knjiga dodaje, mijenja ili briše. Template Method bi omogućio da se zajednički tok obradi na jednom mjestu, dok bi specifični koraci bili izdvojeni.

Observer patern

Observer patern uspostavlja vezu između objekta čije se stanje prati i objekata koji trebaju biti obaviješteni kada dođe do promjene tog stanja. Ovaj patern je posebno koristan kada jedna promjena u sistemu treba izazvati više različitih reakcija.

U aplikaciji Biblioteka najprirodnija primjena Observer paterna odnosi se na sistem obavještenja. Kada se primjerak knjige vrati i postane dostupan, sistem treba obavijestiti prvog korisnika iz reda rezervacija. Slično tome, kada se približi rok vraćanja posuđene knjige, korisniku se može poslati email opomena. U ovom slučaju promjena stanja posudbe ili primjerka predstavlja događaj, dok sistem za email obavještenja reaguje na taj događaj. Time se logika posudbe i rezervacije ne mora direktno vezati za način slanja obavještenja.

Iterator patern

Iterator patern omogućava prolazak kroz kolekciju objekata bez potrebe da se poznaje njena unutrašnja struktura. Klijent koristi jedinstven način pristupa elementima, bez obzira na to da li se podaci nalaze u listi, nizu, skupu ili nekoj drugoj strukturi.

U kontekstu našeg sistema ovaj patern može biti koristan kod prikaza različitih kolekcija podataka. Katalog knjiga, lista rezervacija, lista posudbi i lista korisnika predstavljaju kolekcije kroz koje sistem često prolazi. Iterator bi omogućio ujednačen način prolaska kroz te podatke, posebno kod prikaza rezultata pretrage, generisanja izvještaja ili prikaza korisnikovih posudbi.

Command patern

Command patern pretvara zahtjev u poseban objekat. Na taj način se akcije mogu evidentirati, odgoditi, ponoviti ili poništiti. Ovaj patern je koristan u sistemima gdje se korisničke akcije trebaju jasno pratiti ili gdje postoji potreba za logovanjem aktivnosti.

U aplikaciji Biblioteka se kao komande mogu posmatrati akcije poput posudbe knjige, rezervacije knjige, produženja posudbe i vraćanja knjige. Svaka od tih operacija ima svoje ulazne podatke, pravila i posljedice po sistem. Ako bi se one izdvojile u posebne command klase, bilo bi lakše evidentirati ko je izvršio koju akciju i kada. Također bi se olakšalo dodavanje dodatnih funkcionalnosti, kao što su audit log ili poništavanje pogrešno evidentirane operacije.

Chain of Responsibility patern

Chain of Responsibility patern omogućava da se zahtjev prosljeđuje kroz niz objekata, pri čemu svaki objekat provjerava da li može obraditi zahtjev ili ga prosljeđuje dalje. Ovaj patern je koristan kada se nad jednim zahtjevom mora izvršiti više provjera.

Primjer u našoj aplikaciji je zahtjev za produženje posudbe. Prije nego što sistem odobri produženje, potrebno je provjeriti više uslova: da li korisnik ima aktivnu posudbu, da li je knjiga već rezervisana od strane drugog korisnika, da li je korisnik prekoračio maksimalan broj produženja i da li rok vraćanja već nije istekao. Svaka od ovih provjera mogla bi biti jedan član lanca. Ako bilo koja provjera ne prođe, zahtjev se odbija, a ako sve prođu, rok posudbe se produžava.

Mediator patern

Mediator patern uvodi centralni objekat koji upravlja komunikacijom između više drugih objekata. Umjesto da objekti direktno zavise jedni od drugih, oni komuniciraju preko posrednika. Time se smanjuje međusobna povezanost klasa i olakšava održavanje sistema.

Ovaj patern se može primijeniti kod složenijih procesa u biblioteci, naročito kod vraćanja knjige. Kada bibliotekar evidentira povrat primjerka, potrebno je ažurirati status primjerka, zatvoriti posudbu, provjeriti da li postoje aktivne rezervacije i eventualno poslati email obavještenje korisniku koji je prvi u redu čeka. Umjesto da svaka klasa direktno poziva više drugih klasa, mediator bi koordinisao cijeli proces i odlučivao kojim redoslijedom se izvršavaju potrebne operacije.

Visitor patern

Visitor patern omogućava dodavanje novih operacija nad postojećim objektima bez izmjene njihovih klasa. Koristan je kada sistem ima stabilnu strukturu objekata, ali se nad njima često žele izvršavati različite dodatne operacije.

U našem sistemu se ovaj patern može koristiti kod generisanja izvještaja. Isti skup objekata, kao što su knjige, korisnici, posudbe i rezervacije, može se obrađivati na različite načine. Administrator može željeti izvještaj o najčitanijim knjigama, bibliotekar izvještaj o trenutno posuđenim knjigama, a sistem izvještaj o zakašnjelim posudbama. Visitor bi omogućio da se ove operacije dodaju bez direktnog mijenjanja osnovnih modela.

Memento patern

Memento patern omogućava čuvanje prethodnog stanja objekta kako bi se ono kasnije moglo vratiti. Ovaj patern je koristan u situacijama gdje postoji mogućnost greške pri izmjeni podataka ili gdje administrator treba imati mogućnost vraćanja prethodne verzije.

U okviru našeg sistema ovaj patern se može primijeniti kod izmjene podataka o knjigama i korisnicima. Prije nego što administrator ili bibliotekar izmijeni podatke o knjizi, sistem može sačuvati prethodno stanje. Ako se naknadno utvrdi da su uneseni pogrešni podaci, moguće je vratiti staru verziju bez ručnog ponovnog unosa. Ovo bi bilo posebno korisno kod važnih podataka kao što su naslov, autor, ISBN ili status primjerka.